

End-to-End Non-Profiled Side-Channel Analysis on Long Raw Traces

Jintong Yu^{1,4,5}, Yuxuan Wang^{1,4,5}, Shiwei Qu^{1,4,5}, Yubo Zhao^{1,4,5}, Yipeng Shi^{1,4,5}, Pei Cao², Xiangjun Lu², Chi Zhang^{1,4,5}(✉), Dawu Gu^{1,4,5}(✉), and
Cheng Hong³

¹ Shanghai Jiaotong University, Shanghai, China

² Viewsource (Shanghai) Technology Company, Shanghai, China

³ Ant Group, Hangzhou, China

⁴ State Key Laboratory of Cryptology, P.O. Box 5159, Beijing, 100878, China

⁵ Shanghai Pudong Institute of Cryptology, Shanghai, China

Abstract. With the advancement of deep learning techniques, Deep Learning-based Non-profiled Side-Channel Analysis (DL-NSCA) can automatically learn and combine features, making it a promising method that can skip the manual and precise selection of Points of Interest (PoIs). Existing DL-NSCA methods assume that the attacker can identify a short leakage interval (usually less than 5000 points) containing PoIs from raw traces (more than 100,000 points) and then feed the leakage interval into the neural network to recover the key. However, in practice, the attacker often faces a black-box scenario with unknown underlying implementations, making locating the short interval from raw traces challenging, especially when masking countermeasures exist. To address this issue, we propose a lightweight end-to-end DL-NSCA model called convWIN-MCR, which consists of a performance-optimizing component, convWIN, and an accelerator component, MCR. It can efficiently process raw traces without the need to manually identify the short leakage interval. On the public dataset ASCADv1, while the state-of-the-art model Multi-Output Regression (MOR) requires 28,000 traces and 24 minutes to recover the key from the leakage interval with 1,400 feature points, our framework only requires 6,000 traces in 13 minutes to directly analyze raw traces with 250,000 feature points. To further validate the practical applicability of our framework, we successfully crack a commercial USIM card by analyzing its raw traces and recovering its 128-bit AES key.

Keywords: Non-profiled side-channel analysis · End-to-end framework · Lightweight neural networks.

1 Introduction

Side-Channel Analysis (SCA), first introduced in [10], recovers secrets of cryptographic algorithms by exploiting side-channel information leaked from real-world devices. Based on the attacker’s capability, SCA methods can be categorized into two main types: profiled SCA and non-profiled SCA. Profiled SCA assumes the

attacker has access to a profiling device identical to the target device. On the profiling device, the attacker characterizes the side-channel behaviors and then leverages them to recover the secret key from the target device. Under this condition, profiled SCA is regarded as one of the most powerful methods [14]. In practice, obtaining a fully controlled device that is the same as the target device is difficult, especially for closed devices (such as USIM cards). Non-profiled SCA releases this assumption, only requiring the collection of side-channel traces and corresponding plaintexts (or ciphertexts) from the target device. Therefore, it is more applicable in analyzing many practical scenarios [13,19].

Non-profiled SCA takes a divide-and-conquer strategy, splitting the key into multiple subkeys and analyzing them one by one. For AES-128, the 128-bit key is split into 16 bytes, and each is attacked individually. For each byte, the method tests all 256 key guesses by comparing correlations between side-channel traces and hypothetical leakages related to each key guess to identify the correct key guess by the highest correlation. Traditional non-profiled SCA methods utilize statistical tools to measure this correlation, such as Pearson correlation coefficients (i.e., Correlation Power Analysis, CPA) [3] and mutual information (i.e., Mutual Information Analysis, MIA) [22]. With more countermeasures proposed [1,9], traditional methods struggle to handle side-channel traces without known underlying implementation directly. In 2019, Timon proposed Differential Deep Learning Analysis (DDLA), first introducing deep learning techniques into non-profiled SCA [20]. Instead of statistical tools, DDLA exploits neural networks to learn the correlation between side-channel traces and hypothetical leakages. The advantage of neural networks lies in their ability to automatically learn the positions of Points of Interest (PoIs) and skip the feature selection process.

1.1 Motivations

Existing Deep Learning-based Non-profiled Side-Channel Analysis (DL-NSCA) models are all aimed at the **leakage interval**, which refers to a short interval (or the concatenation of several intervals) that contains PoIs. The leakage interval datasets in their experiments contain less than 5000 feature points. However, locating the leakage interval from raw traces directly without knowing the implementation details is the most challenging part of practical SCA. In real-world scenarios, the attacker can only locate a vague interval based on the type of algorithm, such as identifying the first round operation of AES based on empirical knowledge or trace segmentation techniques [23,24]. This often involves dealing with raw traces with more than 100,000 feature points, especially for software implementations, which we refer to as the **end-to-end** or **black-box** scenario. Processing raw traces is beneficial, especially against masking countermeasures, but the high dimensionality data often leads to poor performance or failed attacks for the following two reasons.

Firstly, existing DL-NSCA models perform low attack performance and have a long analysis time on the leakage interval, which hinders further expansion of the model’s ability to process raw traces. Although DDLA performs better

than traditional non-profiled SCA methods, the computational complexity and runtime are substantial. For instance, to recover a single-byte key, DDLA necessitates training 256 networks. To improve DDLA, Kwon et al. [11] proposed a parallel network and shared layers to reduce the attack time. Moreover, Do et al. [4] proposed the Multi-Output Regression (MOR) network, which outperforms DDLA and Kwon’s method by predicting 256 outputs with a single regression model, reducing computation and easing the constraints of partition-based attacks [18,25]. However, on the leakage interval, MOR typically requires over 10,000 traces and more than 20 minutes for analysis, indicating room for improvement in attack performance and analysis time.

Secondly, existing end-to-end profiled SCA frameworks cannot be directly applied to end-to-end non-profiled SCA. Masure et al. designed a convolutional neural network to handle large-scale raw traces for polymorphic AES implementations [16]. Lu et al. proposed an end-to-end profiled deep learning framework using a deep Long Short-Term Memory (LSTM) network with an attention mechanism, which successfully breaks raw traces protected by first-order masking [2]. They comprise complex networks with multiple layers, and [2] takes three days to recover a 1-byte key. However, complex neural networks can lead to overfitting, as the model learns not only the mapping relationship between the hypothetical leakages related to correct key guess and traces but also the mapping relationship between hypothetical leakages of incorrect key guesses and traces, leading to failure analysis. In addition, its training cost is 256 times higher or more than the profiling setting because non-profiled SCA requires training 256 networks, making the required computational resources and analysis time unacceptable.

1.2 Contribution

In this paper, we propose a lightweight end-to-end model to address the above issues, and our contributions are as follows:

1. **Introducing an efficient neural component into DL-NSCA.** We introduce the convWIN layer, as the first layer of the network. We verify that adding convWIN to DDLA and MOR (State-Of-The-Art, SOTA) can recover keys with 76% fewer traces on average across multiple public datasets. It also enhances the robustness of the models against misalignment.
2. **A framework for lightweight end-to-end DL-NSCA:** We propose a lightweight end-to-end DL-NSCA framework, convWIN-MCR, containing a performance-optimizing component convWIN and an accelerator component MCR. It consists of four layers, enabling the attacker to recover the key in a short runtime in the end-to-end scenario. Our framework eliminates the need for the underlying implementation and manual identification of the leakage interval, requiring only the raw traces and corresponding plain-texts/ciphertexts.
3. **Satisfying results on public datasets and a real-world device:** Our model is capable of directly analyzing raw traces with over 100,000 feature points, and the analysis target includes public datasets ASCADv1 and a

USIM card from a mainstream operator. On a public dataset, MOR needs 28,000 traces and 24 minutes to recover the key from 1,400 selected points. In contrast, our framework only requires 6,000 traces in 13 minutes to analyze raw traces with 250,000 feature points. To demonstrate its real-world applicability, we also successfully crack a commercial USIM card by recovering its 128-bit AES key through direct analysis of raw traces.

1.3 Structure

The paper is organized as follows: Section 2 provides background information. In Section 3, we introduce our end-to-end framework, explaining the motivations behind its design and elucidating the effectiveness of our architecture. In Section 4, we validate the role of the convWIN layer in NSCA and experiment end-to-end analysis on public datasets and a USIM card. Finally, we conclude our work and discuss some potential research directions in Section 5.

2 Background

2.1 Notation

We use calligraphic letters to represent sets like $\mathcal{X}$, with bold upper-case letters (e.g., $\mathbf{X}$) denoting matrices, bold lowercase letters (e.g., $\mathbf{x}$) representing vectors, and lowercase letters (e.g., x) indicating the scalar value. We present functions using a sans serif font, such as f . We target recovering a 1-byte key of AES for each training in this paper, and there are 256 key guesses, i.e., the key space $\mathcal{X} = \{0, 1, \dots, 255\}$. k_* represents the correct key (correct key guess).

2.2 Deep Learning Techniques

Loss Function The goal of a neural network is to learn the mapping relationship f between input set $\mathcal{X}$ and output set $\mathcal{Y}$, and we define the space $\mathcal{F}$ as the set of mapping relationships f : $\mathcal{F} = \{f|\mathcal{Y} = f(\mathcal{X})\}$. In this case, space $\mathcal{F}$ is usually a family of functions determined by a network’s parameter vector $\boldsymbol{\Theta}$: $\mathcal{F} = \{f_{\boldsymbol{\Theta}}|\mathcal{Y} = f_{\boldsymbol{\Theta}}(\mathcal{X}), \boldsymbol{\Theta} \in \mathbb{R}^n\}$. The parameter vector $\boldsymbol{\Theta}$ takes values in the n -dimensional Euclidean space $\mathbb{R}^n$, which is called the parameter space. We set a training dataset of N samples $\mathcal{T} = \{(x_1, y_1), (x_2, y_2), \dots, (x_N, y_N)\}$.

For a given input x_i , a loss function L is used to measure the difference between the output prediction value $f_{\boldsymbol{\Theta}}(x_i)$ and the label y_i , denoted as $L(y_i, f_{\boldsymbol{\Theta}}(x_i))$. According to the empirical risk minimization strategy, the problem of finding the optimal model in the model space $\mathcal{F}$ equivalent to solving the optimization problem: $\min_{\boldsymbol{\Theta}} \frac{1}{N} \sum_{i=1}^N L(y_i, f_{\boldsymbol{\Theta}}(x_i))$. Mean Squared Error (MSE) is a common loss function [7]. MSE measures the average squared difference between the predicted and actual values:

$$\text{MSE} = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2, \quad (1)$$

where y_i denotes the label value, $\hat{y}_i$ denotes the predicted value.

Batch Matrix Multiplication Batch matrix multiplication performs multiple matrix multiplications simultaneously. It is useful when numerous matrix multiplications with the same dimensions are needed. In computer vision, it helps apply transformations and projections to image features in parallel, speeding up processing for multiple images [5,21]. For time series tasks, it is commonly used to multiply hidden states with inputs efficiently [6,15].

Let $\mathbf{A}$ be a three-dimensional tensor of shape (N, M, K) , and $\mathbf{B}$ be a three-dimensional tensor of shape (N, K, P) , where N is the batch size. We define the batch matrix multiplication calculator as $\otimes$. We have

$$\mathbf{A} \otimes \mathbf{B} = (\mathbf{A}[0] \cdot \mathbf{B}[0], \mathbf{A}[1] \cdot \mathbf{B}[1], \dots, \mathbf{A}[N-1] \cdot \mathbf{B}[N-1]),$$

where the dot product (matrix multiplication) is performed between $\mathbf{A}[i]$ and $\mathbf{B}[i]$, $i \in \{0, 1, \dots, N-1\}$. The result of the batch matrix multiplication $\mathbf{A} \otimes \mathbf{B}$ is a three-dimensional tensor of shape (N, M, P) .

2.3 DL-NSCA

Below we introduce the first DL-NSCA work DDLA and the SOTA work MOR.

Differential Deep Learning Analysis (DDLA) [20] Instead of calculating the correlation for every key guess $k \in \mathcal{K}$, Timon trains a neural network $\mathbf{f}_{\boldsymbol{\theta}_k}$ for classification with side-channel traces $\mathcal{T} = \{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_N\}$ as input and the hypothetical leakage $\mathcal{H}_k = \{h_{k,1}, h_{k,2}, \dots, h_{k,N}\}$ as label. The author adopts the MSE loss mentioned in (1). Sort the loss values for each model to derive the correct key guess k_* :

$$k_* = \arg \min_{k \in \mathcal{K}} \frac{1}{N} \sum_{i=1}^N \mathcal{L}(h_{k,i}, \mathbf{f}_{\boldsymbol{\theta}_k}(\mathbf{x}_i)) = \arg \min_{k \in \mathcal{K}} \frac{1}{N} \sum_{i=1}^N (h_{k,i} - \mathbf{f}_{\boldsymbol{\theta}_k}(\mathbf{x}_i))^2.$$

Multi-Output Regression (MOR) [4] MOR has made an improvement in the following three aspects:

1. MOR consists of a shared layer for common feature extraction, followed by 256 non-shared layers, each producing a separate output.
2. DDLA fails under the identity model, as partition-based DPA cannot handle injective target functions [18,25]. MOR addresses this by treating labels as continuous values and applying regression, making it the first effective DL-NSCA method in this context.
3. Unlike DDLA, which trains 256 separate networks, MOR uses a multi-output network with 256 continuous outputs. It maps side-channel traces $\mathcal{T} = \{\mathbf{x}_1, \dots, \mathbf{x}_N\}$ to hypothetical leakages $\mathcal{H} = \{\mathbf{h}_1, \dots, \mathbf{h}_N\}$, where each $\mathbf{h}_i = (h_{0,i}, \dots, h_{255,i})$. The model is trained using MSE loss, and the correct key guess k_* is determined by minimizing the average loss:

$$k_* = \arg \min_{k \in \mathcal{K}} \frac{1}{N} \sum_{i=1}^N \mathcal{L}(\mathbf{h}_i^k, \mathbf{f}_{\boldsymbol{\theta}}^k(\mathbf{x}_i)) = \arg \min_{k \in \mathcal{K}} \frac{1}{N} \sum_{i=1}^N (\mathbf{h}_i^k - \mathbf{f}_{\boldsymbol{\theta}}^k(\mathbf{x}_i))^2$$

where $f_{\theta}^k(x_i)$, $\mathbf{h}_i^k$ respectively represent the k th value of the model output $f_{\theta}(\mathbf{x}_i)$ and label $\mathbf{h}_i$.

2.4 Hypothetical Power Model

Given the intermediate values, hypothetical power models are used to estimate the real power consumption. There are three commonly used models as follows:

- **Hamming Weight(HW) model:** For $a \in \mathbb{F}_2^n$, the HW is defined as
$$\text{HW}(a) = \sum_{i=0}^{n-1} \mathbf{l}_{a[i]=1},$$
 where $\mathbf{l}$ is an indicator function and $a[i]$ denotes the i -th bit of a .
- **Least Significant Bit(LSB) model:** For $a \in \mathbb{F}_2^n$, the LSB model is defined as $\text{LSB}(a) = a[0]$, where $a[0]$ represent the 0th bit of a . Different from the HW model, the LSB model only considers the least significant bit of the data.
- **Identity (ID) model:** For $a \in \mathbb{F}_2^n$, the ID model uses $\text{ID}(a) = a$.

3 End-to-end Framework

3.1 Design Philosophy

Although there are no end-to-end works in non-profiled SCA, two works in profiled SCA use two types of neural network frameworks. The first category is the VGG-like architecture proposed by Masure et al. [16], which is also usually the optimal architecture in the leakage interval scenario [26,27,8]. The second category is the LSTM framework proposed by Lu et al. [14]. The commonality between the two frameworks is that they both utilize neural networks with a high number of layers.

The design suitable for the end-to-end non-profiled framework is significantly different from that of profiled SCA. Our design philosophy for DL-NSCA is to adopt a lightweight network. The complex networks mentioned above are difficult to implement in DL-NSCA for several reasons:

- (1) In end-to-end non-profiled SCA, complex neural network architectures may lead to significant overfitting issues. The main reason lies in the network’s dual behavior. While effectively extracting PoIs in raw traces and establishing correct label-to-trace mappings, it aggregates irrelevant noise points from raw traces simultaneously. These noise points form high-level representations through hierarchical feature composition, ultimately causing the network to erroneously establish spurious correlations between irrelevant features and labels of incorrect key guesses.
- (2) For the same network architecture, the network parameters in profiled SCA will be amplified by a factor in non-profiled SCA. For example, training the profiling model in [14] to recover a 1-byte key can take three days. Therefore, it cannot perform well in DL-NSCA since training this framework for all key

guesses (256 for 1-byte key) would require an unimaginable amount of time and computational resources (at least $3 \times 256 = 768$ days on NVIDIA RTX 2080Ti GPU).

Therefore, the lightweight network should have the following features: (a) enhances performance by significantly improving the representation capability of high-noise raw traces without causing overfitting; (b) uses optimized operators to ensure acceptable analysis time.

We propose a lightweight end-to-end DL-NSCA framework, convWIN-MCR, as shown in Figure 1. It comprises a performance-optimizing component, convWIN, and an accelerator component, MCR. We will introduce them in Section 3.2 and Section 3.3 respectively. We define this framework in Equation (2), where $\gamma_{c_1,1}$ is a convolutional layer with c_1 filters of size 1, β denotes a batch

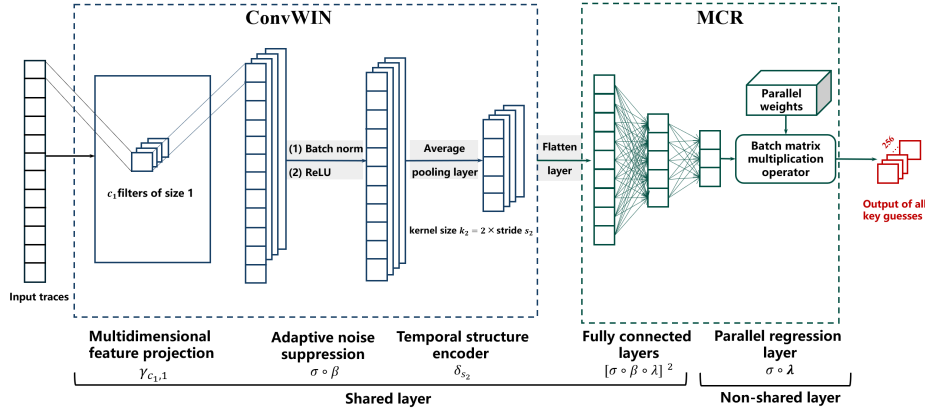


Fig. 1: End-to-end framework of convWIN-MCR

normalization layer, σ represents an activation function ReLU, δ_{s_2} denotes an average pooling layer with kernel size that is twice of the stride size s_2 , λ is the fully connected layer and λ denotes a parallel regression layer.

$$\underbrace{\sigma \circ \lambda}_{\text{non-shared layer}} \circ \underbrace{[\sigma \circ \beta \circ \lambda]^2 \circ [\delta_{s_2} \circ \sigma \circ \beta \circ \gamma_{c_1,1}]}_{\text{shared layer}}, \quad (2)$$

3.2 The Performance-optimizing Component: ConvWIN Layer

To handle long raw traces, we aim to propose a structure that allows the existing model to enhance attack performance while reducing computational complexity. We introduce a ‘temporal feature distiller’ called convWIN as the network’s first layer. It consists of three parts:

- (a) **The multidimensional feature projection $\gamma_{c_1,1}$:** We use a 1×1 convolutional layer, first introduced in the Network in Network (NiN) architecture

by Lin et al. [12]. This layer helps learn independent leakage patterns like clock edges and bus inversions by using different kernels, providing multi-dimensional features for subsequent fully connected layers and enhancing feature interaction.

- (b) **Adaptive noise suppression** $\sigma \circ \beta$: Batch normalization reduces noise and speeds up training, and the ReLU activation sparsify feature representations, adds non-linearity and increases sensitivity to important information.
- (c) **Temporal structure encoder** δ_{s_2} : In machine learning, features are the measurable properties of a dataset. Those useful for the learning task are called relevant features, while the rest are irrelevant [28]. In SCA, relevant features typically refer to PoIs. In long raw traces, PoIs show three key patterns:
 - Few relevant features: Only a small portion of the trace contains useful information, often tied to specific operations like the SBox operation in AES.
 - Local concentration: Relevant features tend to cluster within specific time windows instead of being uniformly spread.
 - Coexistence of strongly and weakly relevant features: Strongly relevant features usually occur precisely when the intermediate value is directly involved in a computation or is actively being processed by a specific hardware unit. In addition, an intermediate value doesn't just appear; it moves through the chip's data path, with each gate or register flip consuming power. These propagated leakages, though less intensive than the strongly relevant features, still carry information and appear alongside the strong ones. We call them weakly relevant features.

We refer to these three characteristics as the sparsity of side-channel leakages. Based on the sparsity, using an average pooling layer as an encoder allows for maximizing the retention of leakage while filtering out noise and significantly reducing feature dimensions, thereby greatly decreasing the required number of parameters.

In addition, for misaligned traces, we demonstrate that applying the average pooling module can significantly improve the correlation of side-channel traces and hypothetical leakages corresponding to the correct key guess. Assume there is a main leakage point P_l at time l within the sequence of power consumptions $P_{-a}, P_{-a+1}, \dots, P_a$, where l follows a uniform distribution in the set $\{-a, -a+1, \dots, a\}$. Let the hypothetical leakages corresponding to the correct key guess as H_{ck} . Denote the Pearson correlation coefficient between P_i and P_j as $\beta_{i,j}$, where $\beta_{i,j} = \rho(P_i, P_j) = \frac{Cov(P_i, P_j)}{\sqrt{Var(P_i)Var(P_j)}}$, $i, j \in \{-a, -a+1, \dots, a\}$. $|\beta_{i,j}| \leq 1$, and the equality holds if and only if P_i is linearly related to P_j . Since the leakage occurs at P_l , there exists α_i , $\alpha_i \leq 1$, s.t. $\rho(H_{ck}, P_i) = \alpha_i \rho(H_{ck}, P_l)$, $i \in \{-a, -a+1, \dots, a\}$.

1. Without the average pooling layer: Denote C_0 as the correlation of misaligned traces and hypothetical leakages corresponding to the correct key guess, and

the expectation of C_0 can be expressed as follows:

$$E[C_0] = E[\rho(H_{ck}, P_l)] = \frac{1}{2a+1} \sum_{i=-a}^a \rho(H_{ck}, P_i) = \frac{\sum_{i=-a}^a \alpha_i}{2a+1} \rho(H_{ck}, P_l).$$

2. Applying the average pooling layer: The output of the pooling layer is $\frac{1}{2a+1} \sum_{i=-a}^a P_i$. The correlation of misaligned traces and hypothetical leak-ages corresponding to the correct key guess C_1 is:

$$C_1 = \rho(H_{ck}, \frac{1}{2a+1} \sum_{i=-a}^a P_i) = \rho(H_{ck}, \sum_{i=-a}^a P_i) = \frac{Cov(H_{ck}, \sum_{i=-a}^a P_i)}{\sqrt{Var(H_{ck})Var(\sum_{i=-a}^a P_i)}}$$

Assuming that the variance of $P_{-a}, P_{-a+1}, \dots, P_a$ are all σ^2 , we have

$$\begin{aligned} C_1 &= \frac{\sigma \sum_{i=-a}^a \alpha_i}{\sqrt{Var(\sum_{i=-a}^a P_i)}} \rho(H_{ck}, P_l) \\ &= \frac{\sum_{i=-a}^a \alpha_i}{\sqrt{(2a+1) + \sum_{i=-a, j=-a, i \neq j}^a \beta_{i,j}}} \rho(H_{ck}, P_l) \\ &\geq \frac{\sum_{i=-a}^a \alpha_i}{\sqrt{(2a+1) + (2a+1)2a}} \rho(H_{ck}, P_l) = \frac{\sum_{i=-a}^a \alpha_i}{2a+1} \rho(H_{ck}, P_l) \\ &= E[C_0] \end{aligned}$$

We have $E[C_0] = C_1 \Leftrightarrow \beta_{i,j} = 1, i, j \in \{-a, -(a-1), \dots, a\}$, where only when the power consumption corresponding to all $(2a+1)$ points are linearly correlated, it is unnecessary to adopt the average pooling strategy. However, this situation rarely occurs in practice. Therefore, adopting the average pooling strategy can improve the probability of distinguishing the correct key guess from incorrect key guesses, thereby enhancing the model's attack performance.

3.3 The Accelerator Component: MCR Layer

One of the main reasons existing DL-NSCA models have difficulty processing long raw traces is that low GPU utilization leads to long analysis time. Based

on MOR, we propose an accelerator component, MCR, which improves GPU utilization by introducing batch matrix multiplication.

DDLA requires training a model for each key guess. The improvement of MOR is that only one network needs to be trained, but there are 256 non-shared layers trained in serial. MCR enables 256 non-shared layers to compute in parallel by merging several matrix multiplications into a single tensor operation.

From serial to parallel Given the input $\mathbf{x} \in \mathbb{R}^{N \times S_{k-1}}$ of the k th non-shared layer, there are 256 fully connected layer, each of which corresponds to one output. Set time stamp $\mathcal{T} = \{t_0, t_1, \dots, t_{255}\}$. The existing serial network performs the following computation at time $t_j, t_j \in \mathcal{T}$:

$$\mathbf{z}_k = \mathbf{x} \mathbf{w}_k^T + b_k$$

where $\mathbf{w}_k^T \in \mathbb{R}^{S_{k-1} \times S_k}$, $b_k \in \mathbb{R}$ are the k th fully connected parameters and $\mathbf{z}_k \in \mathbb{R}^{N \times S_k}$ is the k th output. We introduce batch matrix multiplication to merge 256 matrix multiplications into one single tensor operation. We refer to the newly added dimension as channels, where each channel represents the network training for each key guess. At one time stamp, there are the following computations:

$$\mathbf{Z}^{parallel} = \mathbf{X}^{parallel} \otimes \mathbf{W}^T + \mathbf{b},$$

where $\mathbf{b} = (b_0, b_1, \dots, b_{255}) \in \mathbb{R}^{256}$, $\mathbf{W}^T = (\mathbf{w}_0^T, \mathbf{w}_1^T, \dots, \mathbf{w}_{255}^T) \in \mathbb{R}^{256 \times S_{k-1} \times S_k}$ and $\mathbf{X}^{parallel} = (\mathbf{x}, \mathbf{x}, \dots, \mathbf{x}) \in \mathbb{R}^{256 \times N \times S_{k-1}}$. $\mathbf{Z}^{parallel} = (\mathbf{z}_0, \mathbf{z}_1, \dots, \mathbf{z}_{255})$ is the output. $\otimes$ represents batch multiplication calculator. The structure of the MCR model is shown in the right box of Figure 1. In the parallel regression layer, each color corresponds to a different channel, i.e., one of the key guesses. The calculations in each channel are independent and are performed in parallel during the forward and backward passes of the neural network.

Remark MOR and MCR are functionally equivalent, which means that given the same input and identical internal parameters, they should produce the same output and have the same computational complexity. MCR merges 256 matrix multiplications into a single tensor operation, which significantly reduces the analysis time.

4 Experimental Results

In this section, we first introduce public datasets ASCADv1 in Section 4.1 and validate the role of the convWIN layer in Section 4.2. Section 4.3 and 4.4 are end-to-end experiments on public datasets and a real-world device, respectively. We set the maximum number of training epochs to 50 across all experiments and test the minimum number of traces required using steps of 500. Each model is trained independently 10 times to reduce randomness, such as weight initialization. We differentiate between correct and incorrect key guesses based on the

loss value at the last epoch. In our results, there are two scenarios: (1) When the model successfully recovers the key, i.e., average key rank equals zero in ten times experiments, we find the minimum number of traces to evaluate attack performance. (2) When the model fails to break the target, the performance is measured by the average key rank given the maximum number of traces, denoted as GE. We follow the same pre-processing as in DDLA: normalize the input traces by subtracting the mean and scaling them to the range -1 and 1 . All networks were implemented using the PyTorch framework with an AMD EPYC 7763 64-Core Processor, an NVIDIA GeForce RTX 4090 GPU card, and 1T RAM. The code for replicating the experiments can be found in our GitHub repository⁶.

4.1 ASCADv1 Dataset

The ASCADv1 dataset is a public SCA dataset⁷. It contains traces collected from an 8-bit ATmega8515 microcontroller running masked AES-128 encryption, using Boolean masking [1,17]. ASCADv1 is split into two versions: ASCAD_F (fixed key) and ASCAD_V (variable keys). Since non-profiled SCA targets a device under the same key, we use attack phase traces from ASCAD_V in our experiments (see Table 1). Each dataset provides traces for four types of scenarios. Taking the ASCAD_F dataset as an example, ASCAD_F, ASCAD_F_desyn, ASCAD_F_R and ASCAD_F_R_desyn represent aligned and misaligned traces in the leakage intervals scenario and aligned and misaligned raw traces respectively. For the misaligned traces, the datasets randomly shift the aligned traces within an interval range from the original dataset to simulate random delays.

Table 1: Settings of the ASCAD datasets **Table 2: Settings of Network Parameters**

	Dataset	Traces	Feature points	Random delay
Leakage intervals	ASCAD_F	60,000	700	None
	ASCAD_V	100,000	1,400	None
	ASCAD_F_desyn	50,000	700	50
	ASCAD_V_desyn	100,000	1,400	50
Raw traces	ASCAD_F_R	60,000	100,000	None
	ASCAD_V_R	100,000	250,000	None
	ASCAD_F_R_desyn	60,000	100,000	128
	ASCAD_V_R_desyn	100,000	250,000	128

	DDLA [20]	convWIN-DDLA	MOR [4]	convWIN-MOR
convWIN layer	\	c_1, s_2	\	c_1, s_2
hidden layer 1	20	20	800	800
hidden layer 2	10	10	1000	1000
hidden layer 3	9/2	9/2	1	1
learning rate	0.001	0.001	0.001	0.001
batch size	1000	1000	50	50
loss function	MSE	MSE	MSE	MSE
optimizer	Adam	Adam	Adam	Adam

4.2 The Role of the ConvWIN Layer in NSCA

In this section, we validate the role of the convWIN component in enhancing attack performance and improving robustness against misalignment in the leakage interval scenario. Additionally, the convWIN component has two hyperparameters, and we demonstrate how these internal hyperparameters influence model performance through experiments. Based on DDLA and MOR, we propose the convWIN-DDLA and convWIN-MOR models to explore the impacts of the convWIN component. It is acknowledged that fine-tuning the model can enhance performance for a given dataset; however, finding the optimal parameters for

⁶ https://github.com/jintongyucrypto/end2end_DL-NSCA.git

⁷ available on GitHub: <https://github.com/ANSSI-FR/ASCAD>

a specific scenario is not the main objective of this work. Therefore, we adopt the same model structure and parameters as the original papers, as shown in Table 2. DDLA consists of three fully connected layers, where the hidden size of the last layer depends on the hypothetical power model used (9 for HW, 2 for LSB). MOR comprises two fully connected layers in the shared part and one in the non-shared part. Since it is a regression model, the number of output neurons of the last layer is set to 1. Regarding the hidden size of the shared layers, the authors found that a larger hidden size leads to better model performance. Therefore, we set the hidden size of the shared layers uniformly to 800 and 1000, which will be used for experiments in this section on leakage intervals and in the next section on end-to-end scenarios.

Table 3: Attack performance on aligned and misaligned traces

Dataset	Hypothetical model	DDLA [20]	convWIN-DDLA	MOR [4]	convWIN-MOR
ASCAD_F	HW	GE57.5	22,500	8500	6000
	ID(LSB)	33,000	16,500	24,000	19,500
ASCAD_V	HW	GE100.6	35,500	28,000	4,000
	ID(LSB)	36,000	9,000	GE3.6	34,000
ASCAD_F_desyn	HW	GE75.1	32,500	GE21.2	12,500
	ID(LSB)	GE111.0	4,500	GE5.0	46,500
ASCAD_V_desyn	HW	GE112.8	20,500	GE18.6	4,500
	ID(LSB)	GE100.3	3500	GE16.8	31,000

Performance improvement We evaluate the attack performance of DDLA and MOR with and without the convWIN component, as shown in Table 3. We conduct experiments on the HW and ID models. Since DDLA cannot work in ID model[18], we use the LSB model as a replacement. We analyze ASCADv1 datasets using DDLA, convWIN-DDLA, MOR, and convWIN-MOR models, and the settings of network parameters are shown in Table 2. Here, we discuss the selection of the number of filters c_1 . As mentioned in Section 3.2, c_1 represents the extraction of c_1 leakage patterns (linear transformations) for the subsequent network structure to learn. If we disregard computational complexity, a larger value of c_1 is better. We conduct experiments with $c_1 \in \{1, 2, 4, 8, 16\}$ and a fixed value of s_2 , and the results are shown in Appendix A. As we can see, increasing the number of filters c_1 can improve the model’s performance. The model performance with 1, 2, and 4 filters is significantly worse than that with 8 and 16 filters. Although the model performance corresponding to 16 filters may exceed or equal that of 8 filters in most situations, an increase in the number of filters corresponds to a substantial exponential growth in the number of parameters. Therefore, adopting eight filters represents a trade-off between performance and computational complexity, and we will also utilize $c_1 = 8$ in our subsequent experiments. As shown in Table 3, after adding convWIN to the first layer of DDLA

and MOR, the minimum number of traces required by the model is reduced by an average of 73% and 56%, respectively.

Robustness against misalignment We verify that the addition of the convWIN component can improve the model’s robustness against misalignment, and the results on ASCAD_F_desyn and ASCAD_V_desyn are presented in the Table 3. The stride of the pooling layer s_2 determines the degree of feature dimensionality reduction. A large s_2 significantly reduces the number of network parameters but increases the level of information compression. We consider experimenting with $s_2 \in \{16, 32, 64, 128\}$ and obtain the optimal results. With the addition of the convWIN layer, the performance of DDLA and MOR is improved by an average of 85% and 89%, respectively.

The accelerator component MCR The addition of the convWIN component to the DDLA and MOR models not only significantly reduces the minimum number of traces required for analysis but also decreases the analysis time. This is attributed to the enhanced power of models and the dimensionality reduction effect. Nonetheless, the analysis time for the aforementioned models remains long relative to the dimensionality of the processed traces. When the dimensionality of the input traces increases by a factor of 100, the analysis time becomes unmanageable. Therefore, we replaced the MOR model with our accelerator component MCR, and the experimental results are shown in Table 4. The convWIN-MCR model, while maintaining high performance, improves analysis speed by 95% on aligned traces and by 93% on misaligned traces compared to the convWIN-MOR model. Furthermore, the cost of parallelism, specifically the increased GPU memory consumption, is acceptable. Taking the ASCAD_F dataset as an example, the training process for convWIN-MOR consumes 0.94 GPU memory, and convWIN-MCR’s training occupies 1.03 GB of GPU memory, representing an increase of only 9.6% in GPU memory usage. This allows efficient analysis of raw traces, as reflected in both performance and analysis time. Consequently, we will use the convWIN-MCR framework to analyze the raw traces in subsequent experiments.

4.3 End-to-end Experiments on Public Datasets

Settings for end-to-end analysis We will validate the effectiveness of our end-to-end framework, convWIN-MCR, on raw traces from ASCADv1. The network parameters are shown in Table 2. Since the appropriate stride s_2 depends on the feature dimensionality of the raw traces, we use grid search to find the optimal model. Experiments are conducted on aligned and misaligned raw traces, with the results presented in Table 5. In our experiments, we found that the MOR model fails to recover the key on all raw trace datasets listed in the table. Therefore, to better demonstrate the superiority of our model, we compare the results of MOR on the leakage interval with those of convWIN-MCR on the raw traces on the datasets.”.

Table 4: The role of the accelerator component MCR

Dataset	Model	Hypothetical Model	Minimum number of Traces	Analysis time(second)
ASCAD_F	DDLA[20]	HW/LSB	GE57.5/33000	fail/2581
	convWIN-DDLA	HW/LSB	22500/16500	2343/1569
	MOR[4]	HW/ID	8500/24000	404/1156
	convWIN-MOR	HW/ID	6000 /19500	299/851
	convWIN-MCR	HW/ID	6000 /19500	15 /42
ASCAD_V	DDLA[20]	HW/LSB	GE100.6/36000	fail/5970
	convWIN-DDLA	HW/LSB	35500/9000	10119/2554
	MOR[4]	HW/ID	28000/GE3.6	1401/fail
	convWIN-MOR	HW/ID	4000 /34000	199/1626
	convWIN-MCR	HW/ID	4000 /34000	10 /84
ASCAD_F_desyn	DDLA[20]	HW/LSB	GE75.1/GE111.0	fail/fail
	convWIN-DDLA	HW/LSB	32500/ 4500	8189/1228
	MOR[4]	HW/ID	GE21.2/GE5.0	fail/fail
	convWIN-MOR	HW/ID	12500/46500	918/2571
	convWIN-MCR	HW/ID	12500/46500	62 /170
ASCAD_V_desyn	DDLA[20]	HW/LSB	GE112.8/GE100.3	fail/fail
	convWIN-DDLA	HW/LSB	20500/ 3500	4510/893
	MOR[4]	HW/ID	GE18.6/GE16.8	fail/fail
	convWIN-MOR	HW/ID	4500/31000	245 / 1670
	convWIN-MCR	HW/ID	4500/31000	15 / 101

Results on aligned raw traces On the ASCAD_F_R dataset, the experiment reveals that $s_2 = 200$ leads to the best performance, successfully recovering the key using only 10,000 traces in 3 minutes. Our model’s analysis performance on raw traces approaches MOR’s on the leakage interval with only 700 points, while the analysis time is also reduced by 59%. On the ASCAD_V_R dataset, which contains 250,000 feature points, the experiment shows that the best results are achieved when $s_2 = 1000$. On the leakage interval dataset containing 1400 points, MOR requires at least 28000 traces and 24 minutes to recover the key. Our framework analyzes the raw traces from the same dataset with 250,000 points, requiring only 6000 traces and completing the analysis within 13 minutes.

Results on misaligned raw traces On the ASCAD_F_R_desyn dataset, the convWIN-MCR model performs best when $s_2 = 1000$. It successfully recovers the key using 38500 traces in 10 minutes, while MOR fails to recover the key on the corresponding leakage interval dataset. On the ASCAD_V_R_desyn dataset, the experiments indicate that when $s_2 = 2000$, the optimal value is reached, successfully recovering the key using only 7000 traces in just 6 minutes. MOR fails to recover the key on the corresponding leakage interval dataset with 1400 points.

4.4 End-to-end Experiments on a USIM Card

Raw traces acquisition from a USIM card While deep learning-based SCA methods perform well on public datasets, few have been tested on real-world devices. We evaluate the practicality of our end-to-end framework in a realistic setting by targeting a 4G USIM card from a major mobile operator

Table 5: Attack performance of convWIN-MCR on aligned and misaligned raw traces

Dataset	Number of feature points	Model	Hypothetical Minimum number of traces	Analysis time (second)
ASCAD_F	700	MOR	HW 8,500 ID 24,000	404 1,156
ASCAD_F_R	100,000	convWIN-MCR	HW 10,000 ID 42,500	167 632
ASCAD_V	1400	MOR	HW 28,000 ID GE3.6	1401 fail
ASCAD_V_R	250,000	convWIN-MCR	HW 6,000 ID 50,000	744 1,588
ASCAD_F_desyn	700	MOR	HW GE21.2 ID GE5.0	fail fail
ASCAD_F_R_desyn	100,000	convWIN-MCR	HW 40,000 ID 38,500	544 558
ASCAD_V_desyn	1400	MOR	HW GE18.6 ID GE16.8	fail fail
ASCAD_V_R_desyn	250,000	convWIN-MCR	HW 7,000 ID 33,500	361 1,001

(referred to as Card A). 4G⁸ authentication uses the MILENAGE algorithm, based on the AES algorithm. To break a USIM card, an attacker must recover the master key K and random value OPc . Liu et al. achieved this by analyzing leakages of the SubByte operations in the first two rounds of AES using CPA [13]. Our replication shows that the success of the CPA is heavily dependent on the accurate selection of the leakage interval and alignment. In our experiment, we

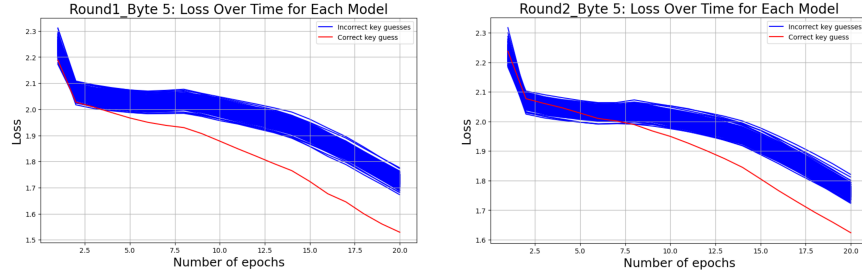


Fig. 2: Results of end-to-end analysis on the USIM card

utilized an acquisition device, the PICO 3203D oscilloscope, and a PC to collect the power traces of the USIM card’s authentication process, as shown in Figure 5 of appendix B. As illustrated in Figure 6, we collect 30,000 traces containing the first two rounds of operations without any additional preprocessing and create a dataset called TRACE_USIM⁹, which includes 100,000 feature points for each round of operations in USIM_1rd and USIM_2rd.

⁸ <https://www.3gpp.org/dynareport?code=33-series.html>

⁹ https://github.com/jintongyucrypto/end2end_DL-NSCA.git

Results of analyzing raw traces collected from USIM card We analyze the raw traces USIM_1rd containing the first round of AES encryption using Liu’s method. We observe that it is impossible to distinguish between the correct key guess and incorrect key guesses. We use our end-to-end framework convWIN-MCR, taking raw traces as the input and the intermediate values $[HW[Sbox[p \oplus 0x00]], HW[Sbox[p \oplus 0x01]], \dots, HW[Sbox[p \oplus 0xFF]]]$ as labels, where p is the known plaintext. The model structure parameters are shown in the last column of Table 2. We perform a grid search on the parameter $s_2 \in \{100, 200, 500, 1000, 2000\}$ and consider that the output component corresponding to the fastest decrease of the loss function indicates the position of the correct key guess. Experimental results show that for all 16 bytes, we can successfully distinguish between the correct and the incorrect key guesses through the loss curve within 6 minutes, and figure 2 shows the loss curve for the 5th byte. Using the early stopping technique, we find that at 20 epochs, the model has already completed distinguishing between the correct and incorrect key guesses. Therefore, the value of $K \oplus OPc$ of this USIM card is recovered.

Next, we analyze raw traces USIM_2rd using the same model structure and the hyperparameter search to recover first round subkey. For all bytes, the loss curve shows strong distinguishability, and we recover the 128-bit round key in 7 minutes. We only show the loss curve of the 5th byte of the key in Figure 2 considering space constraints. Based on the first round subkey, we can finally deduce the master key and OPc .

5 Conclusions

This paper proposes the first efficient end-to-end DL-NSCA framework, convWIN-MCR. This framework allows attackers to recover keys in a black-box scenario without owning a profiling device and knowing the implementation details. We get satisfying results on public datasets and a real-world device. On the public dataset, the number of feature points in the raw traces exceeds that of the leakage interval by more than two orders of magnitude. We demonstrate that convWIN-MCR achieves comparable or even superior performance in key recovery when analyzing the raw traces, compared to MOR which analyzes the leakage interval. Moreover, our framework significantly reduces analysis time. On a real-world device, we validate the practicality of convWIN-MCR in realistic black-box scenarios by successfully recovering the master key through direct analysis of raw traces from a USIM card.

Furthermore, our experiments utilize public datasets primarily employing first-order masking, as this masking scheme has minimal impact on algorithm performance and is the most widely used. However, higher-order masking implementations with higher security levels hold significant practical importance for future research.

Acknowledgments. This work was supported by the National Natural Science Foundation of China (Grant No.U2336210), the Ant Group, and the Startup Fund for Young Faculty at SJTU (SFYF at SJTU).

References

1. Akkar, M.L., Giraud, C.: An implementation of des and aes, secure against some attacks. In: Cryptographic Hardware and Embedded Systems—CHES 2001: Third International Workshop Paris, France, May 14–16, 2001 Proceedings 3. pp. 309–318. Springer (2001)
2. Bengio, Y., Courville, A., Vincent, P.: Representation learning: A review and new perspectives. *IEEE transactions on pattern analysis and machine intelligence* **35**(8), 1798–1828 (2013)
3. Brier, E., Clavier, C., Olivier, F.: Correlation power analysis with a leakage model. In: Cryptographic Hardware and Embedded Systems-CHES 2004: 6th International Workshop Cambridge, MA, USA, August 11–13, 2004. Proceedings 6. pp. 16–29. Springer (2004)
4. Do, N.T., Hoang, V.P., Doan, V.S.: A novel non-profiled side channel attack based on multi-output regression neural network. *Journal of Cryptographic Engineering* **14**(3), 427–439 (2024)
5. Fu, J., Liu, J., Tian, H., Li, Y., Bao, Y., Fang, Z., Lu, H.: Dual attention network for scene segmentation. In: Proceedings of the IEEE/CVF conference on computer vision and pattern recognition. pp. 3146–3154 (2019)
6. Graves, A., Graves, A.: Long short-term memory. Supervised sequence labelling with recurrent neural networks pp. 37–45 (2012)
7. Hastie, T., Tibshirani, R., Friedman, J., Franklin, J.: The elements of statistical learning: data mining, inference and prediction. *The Mathematical Intelligencer* **27**(2), 83–85 (2005)
8. Kim, J., Picek, S., Heuser, A., Bhasin, S., Hanjalic, A.: Make some noise. unleashing the power of convolutional neural networks for profiled side-channel analysis. *IACR Transactions on Cryptographic Hardware and Embedded Systems* pp. 148–179 (2019)
9. Kocher, P., Jaffe, J., Jun, B.: Differential power analysis. In: Advances in Cryptology—CRYPTO’99: 19th Annual International Cryptology Conference Santa Barbara, California, USA, August 15–19, 1999 Proceedings 19. pp. 388–397. Springer (1999)
10. Kocher, P.C.: Timing attacks on implementations of diffie-hellman, rsa, dss, and other systems. In: Advances in Cryptology—CRYPTO’96: 16th Annual International Cryptology Conference Santa Barbara, California, USA August 18–22, 1996 Proceedings 16. pp. 104–113. Springer (1996)
11. Kwon, D., Hong, S., Kim, H.: Optimizing implementations of non-profiled deep learning-based side-channel attacks. *IEEE Access* **10**, 5957–5967 (2022)
12. Lin, M., Chen, Q., Yan, S.: Network in network. *arXiv preprint arXiv:1312.4400* (2013)
13. Liu, J., Yu, Y., Standaert, F.X., Guo, Z., Gu, D., Sun, W., Ge, Y., Xie, X.: Small tweaks do not help: Differential power analysis of milenage implementations in 3g/4g usim cards. In: Computer Security—ESORICS 2015: 20th European Symposium on Research in Computer Security, Vienna, Austria, September 21–25, 2015, Proceedings, Part I 20. pp. 468–480. Springer (2015)
14. Lu, X., Zhang, C., Cao, P., Gu, D., Lu, H.: Pay attention to raw traces: A deep learning architecture for end-to-end profiling attacks. *IACR Transactions on Cryptographic Hardware and Embedded Systems* pp. 235–274 (2021)
15. Luong, M.T., Pham, H., Manning, C.D.: Effective approaches to attention-based neural machine translation. *arXiv preprint arXiv:1508.04025* (2015)

16. Masure, L., Belleville, N., Cagli, E., Cornelie, M.A., Couroussé, D., Dumas, C., Maingault, L.: Deep learning side-channel analysis on large-scale traces: A case study on a polymorphic aes. In: European Symposium on Research in Computer Security. pp. 440–460. Springer (2020)
17. Prouff, E., Rivain, M.: A generic method for secure sbox implementation. In: International Workshop on Information Security Applications. pp. 227–244. Springer (2007)
18. Standaert, F.X., Gierlichs, B., Verbauwhede, I.: Partition vs. comparison side-channel distinguishers: An empirical evaluation of statistical tests for univariate side-channel attacks against two unprotected cmos devices. In: International Conference on Information Security and Cryptology. pp. 253–267. Springer (2008)
19. Tihmstar: Using a magic wand to break the iphone’s last security barrier (2022), <https://hardware.io/netherlands-2022/speakers/tihmstar.php>.
20. Timon, B.: Non-profiled deep learning-based side-channel attacks with sensitivity analysis. IACR Transactions on Cryptographic Hardware and Embedded Systems pp. 107–131 (2019)
21. Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, Ł., Polosukhin, I.: Attention is all you need. *Advances in neural information processing systems* **30** (2017)
22. Veyrat-Charvillon, N., Standaert, F.X.: Mutual information analysis: how, when and why? In: International workshop on cryptographic hardware and embedded systems. pp. 429–443. Springer (2009)
23. Wang, A., He, S., Wei, C., Sun, S., Ding, Y., Wang, J.: Using convolutional neural network to redress outliers in clustering based side-channel analysis on cryptosystem. In: International Conference on Smart Computing and Communication. pp. 360–370. Springer (2022)
24. Wang, Z., Ding, Y., Wang, A., Zhang, Y., Wei, C., Sun, S., Zhu, L.: Spa-gpt: general pulse tailor for simple power analysis based on reinforcement learning. *IACR Transactions on Cryptographic Hardware and Embedded Systems* **2024**(4), 40–83 (2024)
25. Whitnall, C., Oswald, E., Standaert, F.X.: The myth of generic dpa... and the magic of learning. In: Topics in Cryptology–CT-RSA 2014: The Cryptographer’s Track at the RSA Conference 2014, San Francisco, CA, USA, February 25–28, 2014. Proceedings. pp. 183–205. Springer (2014)
26. Wouters, L., Arribas, V., Gierlichs, B., Preneel, B.: Revisiting a methodology for efficient cnn architectures in profiling attacks. *IACR Transactions on Cryptographic Hardware and Embedded Systems* pp. 147–168 (2020)
27. Zaid, G., Bossuet, L., Habrard, A., Venelli, A.: Methodology for efficient cnn architectures in profiling attacks. *IACR Transactions on Cryptographic Hardware and Embedded Systems* pp. 1–36 (2020)
28. Zhou, Z.H.: Machine learning. Springer nature (2021)

Appendix A Ablation study on the kernel size c_1

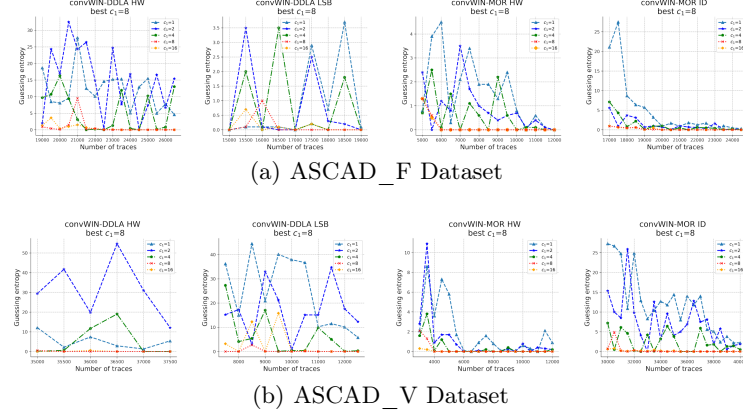


Fig. 3: Attack performance of convWIN-DDLA and convWIN-MOR given different settings of the kernel size c_1

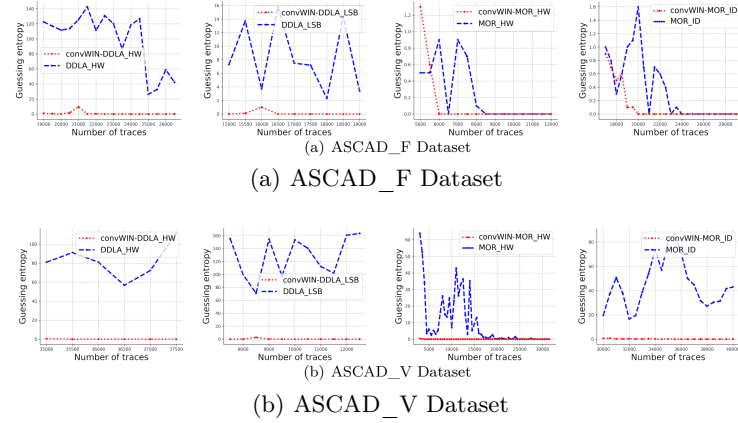


Fig. 4: Performance comparison between DDLA and convWIN-DDLA, MOR and convWIN-MOR

Appendix B The USIM Card Analysis

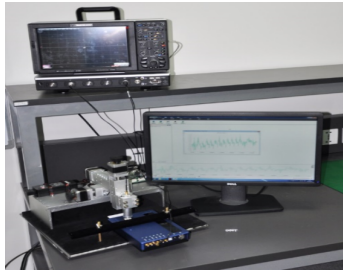


Fig.5: The Acquisition settings for the USIM Card

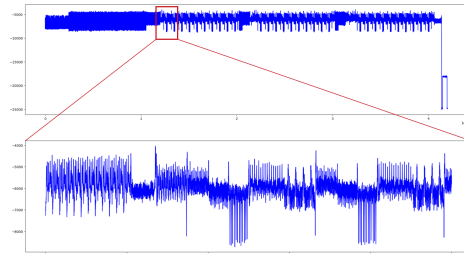


Fig.6: Capturing traces from the first two round of AES encryption